

Práctica 4 ED | 2º Ingeniería Informática David Rodríguez Aparicio

En esta práctica hemos creado una agenda de eventos mediante un Arbol Binario de Busqueda, en el que cada evento se almacena en el árbol de forma cronológica.

En base a los ejemplos y conocimientos del temario he creado el código, que carga un fichero de texto y lo convierte en Eventos, y de estos creamos la agenda de eventos.

Se usa un Iterador con pila, que recorre el arbol con recorrido inorden, también hemos usado recursividad.

EVENTOS.CPP

Aqui vemos la base del programa que es la estructura de los eventos.

```
#include "Eventos.h"

#include <cassert>

#include <sstream>
// #include <iomanip>

using namespace std;

/* ----- CONSTRUCTORES ----- */

Evento::Evento(){
    this->dia = 1;
    this->mes = 1;
    this->anio = 1;
    this->descripcion = " ";
}

Evento::Evento(int dia, int mes, int anio, string descripcion){
    this->dia = dia;
    this->mes = mes;
    this->anio = anio;
    this->descripcion = descripcion;
}

Evento::Evento(const Evento& otroEvento){
    this->dia = otroEvento.dia;
    this->mes = otroEvento.mes;
    this->anio = otroEvento.anio;
    this->descripcion = otroEvento.descripcion;
```

```
}

/* ----- METODOS ----- */

bool Evento::comprobarFecha() const{
    if (this->anio <= 0) return false;

    if (this->mes < 1 || this->mes > 12) return false;

    if (this->dia < 1 || this->dia > 31) return false;

    return true;
}

string Evento::toString() const{
    string texto;

    texto += "Evento: " + this->descripcion;
    texto += "\n Fecha: " + to_string( this->dia ) + "/" + to_string( this->mes )
+ "/" + to_string( this->anio ) + "\n" ;

    return texto;
}

bool Evento::operator<(const Evento& otroEvento) const{

    if (this->anio != otroEvento.anio) return this->anio < otroEvento.anio;
    if (this->mes != otroEvento.mes) return this->mes < otroEvento.mes;
    return this->dia < otroEvento.dia;
}

bool Evento::operator==(const Evento& otroEvento) const{
    return (dia == otroEvento.dia && mes == otroEvento.mes && anio ==
otroEvento.anio);
}

/* ----- SETTERS / GETTERS ----- */

int Evento::getDia() const{
    return dia;
}

int Evento::getMes() const{
    return mes;
}

int Evento::getAnio() const{
    return anio;
}

string Evento::getDescripcion() const{
    return descripcion;
}
```

```

}

void Evento::setDia(int nuevoDia){
    this->dia = nuevoDia;
    assert(comprobarFecha());
}

void Evento::setMes(int nuevoMes){
    this->mes = nuevoMes;
    assert(comprobarFecha());
}

void Evento::setAnio(int nuevoAnio){
    this->anio = nuevoAnio;
    assert(comprobarFecha());
}

void Evento::setDescripcion(string nuevaDescripcion){
    this->descripcion = nuevaDescripcion;
}

```

AGENDAEVENTO.CPP

Y aqui tenemos la agenda, donde implementamos la estructura nodo e Iterador, dentro de la propia clase de AgendaEvento, de esta manera conseguimos abstracción.

Puede que algunos métodos estén pensados para la interacción con el usuario, pero al final no se ha terminado de implementar esta característica (Creo que el enunciado no lo menciona).

```

#include "AgendaEvento.h"

#include <cassert>
#include <string>

using namespace std;

/* ----- Parte Recursiva ----- */

AgendaEvento::Nodo* AgendaEvento::clonarNodo(const Nodo* nodo){
    if (nodo == nullptr) return nullptr;

    Nodo* nuevo = new Nodo(nodo->evento);
    nuevo->HijoIzq = clonarNodo(nodo->HijoIzq);
    nuevo->HijoDer = clonarNodo(nodo->HijoDer);
    return nuevo;
}

```

```
void AgendaEvento::liberarNodo(Nodo* nodo){
    if (nodo == nullptr) return;

    liberarNodo(nodo->HijoIzq);
    liberarNodo(nodo->HijoDer);
    delete nodo;
}

bool AgendaEvento::insertarNodo(Nodo*& nodo, const Evento& evento, bool
actualizarSiExiste){
    if (nodo == nullptr) {
        nodo = new Nodo(evento);
        return true;
    }

    if (evento < nodo->evento) {
        return insertarNodo(nodo->HijoIzq, evento, actualizarSiExiste);
    }
    else if (nodo->evento < evento) {
        return insertarNodo(nodo->HijoDer, evento, actualizarSiExiste);
    }
    else {
        if (actualizarSiExiste) {
            nodo->evento.setDescripcion( evento.getDescripcion() );
            return true;
        }
        return false;
    }
}

AgendaEvento::Nodo* AgendaEvento::buscarNodo(Nodo* nodo, const Evento& clave)
const{
    if (nodo == nullptr) return nullptr;

    if (clave == nodo->evento) return nodo;

    if (clave < nodo->evento)
        return buscarNodo(nodo->HijoIzq, clave);

    return buscarNodo(nodo->HijoDer, clave);
}

int AgendaEvento::alturaNodo(Nodo* nodo) const{
    if (nodo == nullptr) return 0;

    int hi = alturaNodo(nodo->HijoIzq);
    int hd = alturaNodo(nodo->HijoDer);

    return 1 + (hi > hd ? hi : hd);
}
```

```
int AgendaEvento::cantidadNodos(Nodo* nodo) const{
    if (nodo == nullptr) return 0;
    return 1 + cantidadNodos(nodo->HijoIzq) + cantidadNodos(nodo->HijoDer);
}

/* ----- Parte Pública ----- */

/* ----- CONSTRUCTOR / DESTRUCTOR ----- */

AgendaEvento::AgendaEvento(){
    this->raiz = nullptr;
}

AgendaEvento::AgendaEvento(const AgendaEvento& otraAgenda){
    this->raiz = clonarNodo(otraAgenda.raiz);
}

AgendaEvento& AgendaEvento::operator=(const AgendaEvento& otraAgenda){
    if (this == &otraAgenda) return *this;

    liberarNodo(this->raiz);
    this->raiz = nullptr;

    this->raiz = clonarNodo(otraAgenda.raiz);

    return *this;
}

AgendaEvento::~~AgendaEvento(){
    liberarNodo(this->raiz);
    this->raiz = nullptr;
}

/* ----- METODOS ----- */

bool AgendaEvento::insertar(const Evento& nuevoEvento, bool actualizarSiExiste){
    return insertarNodo(this->raiz, nuevoEvento, actualizarSiExiste);
}

bool AgendaEvento::existe(const Evento& clave) const{
    return (buscarNodo(this->raiz, clave) != nullptr);
}

bool AgendaEvento::obtener(const Evento& clave, Evento& out) const{
    Nodo* n = buscarNodo(this->raiz, clave);
    if (n == nullptr) return false;
}
```

```
        out = n->evento;
        return true;
    }

    int AgendaEvento::alturaArbol() const{
        return alturaNodo(this->raiz);
    }

    int AgendaEvento::numNodos() const{
        return cantidadNodos(this->raiz);
    }

    bool AgendaEvento::estaVacia() const{
        return (this->raiz == nullptr);
    }

    /* ----- Parte ITERADOR ----- */

    void AgendaEvento::Iterador::push(Nodo* nodo){
        pila = new PilaNodo(nodo, pila);
    }

    AgendaEvento::Nodo* AgendaEvento::Iterador::pop(){
        if (pila == nullptr) return nullptr;

        PilaNodo* top = pila;
        Nodo* n = top->nodo;

        pila = top->sig;
        delete top;

        return n;
    }

    void AgendaEvento::Iterador::bajarIzquierda(Nodo* nodo){
        Nodo* cur = nodo;
        while (cur != nullptr) {
            push(cur);
            cur = cur->HijoIzq;
        }
    }

    /* ----- Constructores Iterador ----- */

    AgendaEvento::Iterador::Iterador(){
        this->actual = nullptr;
        this->pila = nullptr;
    }

    AgendaEvento::Iterador::Iterador(Nodo* raiz){
```

```
    this->actual = nullptr;
    this->pila = nullptr;

    bajarIzquierda(raiz);
}

AgendaEvento::Iterador::Iterador(const Iterador& otroIterador){
    int count = 0;
    for (PilaNodo* p = otroIterador.pila; p != nullptr; p = p->sig) count++;

    Nodo** arr = nullptr;
    if (count > 0) arr = new Nodo*[count];

    int i = count - 1;
    for (PilaNodo* p = otroIterador.pila; p != nullptr; p = p->sig) {
        arr[i--] = p->nodo;
    }

    this->pila = nullptr;
    for (int k = 0; k < count; k++) {
        this->pila = new PilaNodo(arr[k], this->pila);
    }

    delete[] arr;

    this->actual = otroIterador.actual;
}

AgendaEvento::Iterador::~~Iterador(){
    while (pila != nullptr) {
        PilaNodo* tmp = pila;
        pila = pila->sig;
        delete tmp;
    }
    actual = nullptr;
}

/* Sobrecarga de = */
AgendaEvento::Iterador& AgendaEvento::Iterador::operator=(const Iterador&
iterador){
    if (this == &iterador) return *this;

    // liberar mi pila
    while (this->pila != nullptr) {
        PilaNodo* tmp = this->pila;
        this->pila = this->pila->sig;
        delete tmp;
    }

    // copiar pila del otro
    int count = 0;
    for (PilaNodo* p = iterador.pila; p != nullptr; p = p->sig) count++;
```

```

    Nodo** arr = nullptr;
    if (count > 0) arr = new Nodo*[count];

    int i = count - 1;
    for (PilaNodo* p = iterador.pila; p != nullptr; p = p->sig) {
        arr[i--] = p->nodo;
    }

    this->pila = nullptr;
    for (int k = 0; k < count; k++) {
        this->pila = new PilaNodo(arr[k], this->pila);
    }

    delete[] arr;

    this->actual = iterador.actual;

    return *this;
}

/* ----- Metodos Iterador ----- */

bool AgendaEvento::Iterador::tieneSiguiente() const{
    return (pila != nullptr);
}

Evento AgendaEvento::Iterador::siguienteEvento(){
    Nodo* nodo = pop();
    actual = nodo;

    if (nodo != nullptr && nodo->HijoDer != nullptr) {
        bajarIzquierda(nodo->HijoDer);
    }

    return nodo->evento;
}

AgendaEvento::Iterador AgendaEvento::iterador() const{
    return Iterador(this->raiz);
}

```

MAIN.CPP

Finalmente tenemos el main, creamos una funcion para cargar el archivo y usamos lo creado anteriormente para mostrar la Agenda.

```

#include "AgendaEvento.h"

#include <iostream>

```



```

#include <fstream>
#include <sstream>
#include <string>

using namespace std;

/**
 * @brief Lee y parsea una línea de texto con un evento.
 *
 * Formato esperado por línea:
 *   día mes año descripción...
 * Ejemplo:
 *   12 10 2025 Examen de Estructura de Datos
 *
 * @param linea Línea completa del fichero.
 * @param outEvento Evento de salida (se rellena si el parseo es correcto).
 * @return true si la línea contiene un evento válido a nivel de formato, false si
no.
 */
static bool leerEventoDeLinea(const string& linea, Evento& outEvento){
    istringstream iss(linea);

    int dia, mes, anio;
    if (!(iss >> dia >> mes >> anio)) {
        return false; // línea inválida o vacía
    }

    string descripcion;
    getline(iss, descripcion);

    // Elimina el espacio inicial típico tras leer año y luego hacer getline
    if (!descripcion.empty() && descripcion[0] == ' ') descripcion.erase(0, 1);

    outEvento = Evento(dia, mes, anio, descripcion);
    return true;
}

/**
 * @brief Programa principal de la práctica.
 *
 * - Lee un fichero de eventos (por defecto: "datos/agendaEventos.txt").
 * - Inserta cada evento en un ABB (AgendaEvento) ordenado por fecha.
 * - Muestra información estructural del árbol (nº nodos y altura).
 * - Recorre y muestra los eventos en orden cronológico usando un iterador
inorden.
 *
 * Uso:
 *   practica4.exe [ruta_fichero]
 *
 * Si se proporciona ruta por argumento, se utilizará en lugar de la ruta por
defecto.
 *
 * @param argc Número de argumentos.
 * @param argv Vector de argumentos (argv[1] puede contener la ruta del fichero).

```

```
* @return 0 si finaliza correctamente, 1 si no se puede abrir el fichero.
*/
int main(int argc, char* argv[]){

    string ruta = "datos/agendaEventos.txt";
    if (argc >= 2) {
        ruta = argv[1];
    }

    ifstream fin(ruta);
    if (!fin) {
        cerr << "No se pudo abrir el fichero '" << ruta << "'\n";
        return 1;
    }

    AgendaEvento agenda;

    string linea;
    int numLeidas = 0;
    int numInsertadas = 0;

    while (getline(fin, linea)) {
        // Saltar líneas vacías
        if (linea.size() == 0) continue;

        Evento ev;
        if (!leerEventoDeLinea(linea, ev)) {
            cerr << "línea inválida ignorada: " << linea << "\n";
            continue;
        }

        numLeidas++;

        // Inserta el evento; si ya existe esa fecha, se actualiza
        (actualizarSiExiste = true)
        if (agenda.insertar(ev, true)) {
            numInsertadas++;
        }
    }

    fin.close();

    cout << "Eventos leídos: " << numLeidas << "\n";
    cout << "Eventos insertados: " << numInsertadas << "\n";
    cout << "Nodos en el árbol: " << agenda.numNodos() << "\n";
    cout << "Altura del árbol: " << agenda.alturaArbol() << "\n\n";

    cout << "--- Eventos en orden cronológico --- \n";

    // Recorrido ordenado mediante iterador inorden
    AgendaEvento::Iterador it = agenda.iterador();
    while (it.tieneSiguiente()) {
        Evento ev = it.siguienteEvento();
        cout << ev.toString() << "\n";
    }
}
```

```
    }  
  
    return 0;  
}
```